Tema 7:
Conjuntos y bolsas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

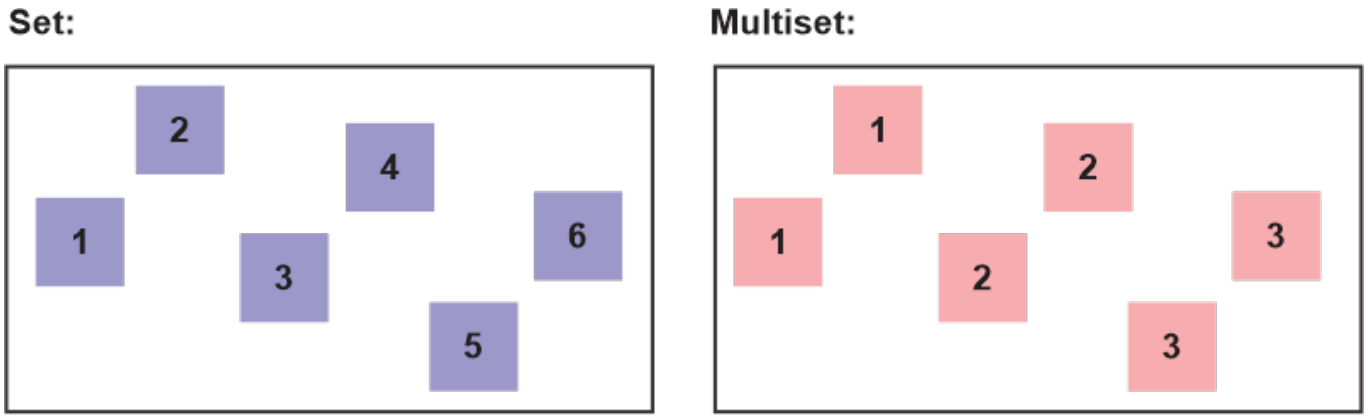
Curso 2023-2024

Contenidos

- Definición
- Los TADs Conjunto y Bolsa
- Implementación
- Conjuntos en Pascal
- Ejercicios propuestos

Conjunto y bolsa: definiciones

- **Conjunto (*set*)**: Estructura de datos no lineal que representa una colección no ordenada de elementos en la que no se admiten duplicados.
- **Bolsa (*multiset*)**: contenedor similar a un conjunto que admite duplicados.



TAD Conjunto

- Independientemente de su implementación, un conjunto proporciona las siguientes 4 operaciones básicas:
  - Añadir un elemento (`add`)
  - Eliminar un elemento (`remove`)
  - Consultar si un elemento está en el conjunto (`in`)
  - Conocer el tamaño o número de elementos (`size`)
  - \+ Inicializar/crear
- Adicionalmente: Unión, Intersección, Diferencia, Extracción de subconjuntos, Inclusión, etc.

TAD Conjunto: especificación

- **Nombre:** Conjunto
- **Valores posibles:** *vacío, otros (según elementos)*.
- **Operaciones:**
  - `add`
  - `remove`
  - `in`
  - `size`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

TAD Bolsa

- Una bolsa proporciona 5 operaciones básicas:
  - Añadir un elemento (`add`)
  - Eliminar un elemento (`remove`)
  - Eliminar todas las ocurrencias de un elemento (`remove_all`)
  - Multiplicidad de un elemento (`multiplicity`)
  - Conocer el tamaño o número de elementos (`size`)
  - \+ Inicializar/crear

TAD Bolsa: especificación

- **Nombre:** Bolsa
- **Valores posibles:** *vacía, otros (según elementos)*.
- **Operaciones:**
  - `add`
  - `remove`
  - `remove_all`
  - `multiplicity`
  - `size`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

Implementación

- Los conjuntos y bolsas pueden implementarse utilizando diversas estructuras de datos, como arrays, listas enlazadas, árboles de búsqueda binarios o tablas hash.
- Implementación en memoria contigua (arrays):
  - Acceso eficiente e inmediato a cualquier elemento mediante acceso aleatorio.
  - El tamaño fijo puede llevar a quedarse sin espacio o a desperdiciar espacio.
- Implementación en memoria dinámica:
  - Almacenamiento eficiente y ajustado al número de nodos
  - Encontrar un nodo concreto requiere recorridos de (potencialmente) muchos otros nodos precedentes → O(n)

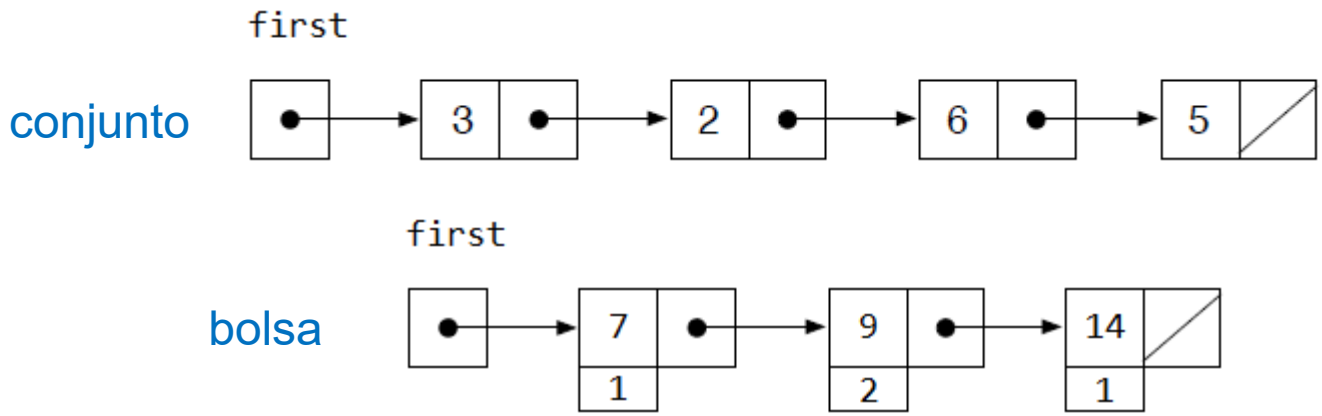
Implementación mediante arrays

- Se define un array cuyos índices son los posibles valores que pueden incluirse en el conjunto.
- Limitado a tipos base que permitan indexar un array: generalmente enteros y otros ordinales.
  - Este es el enfoque del tipo Set en Pascal

F	F	F	T	T	T
0	1	2	3	4	5

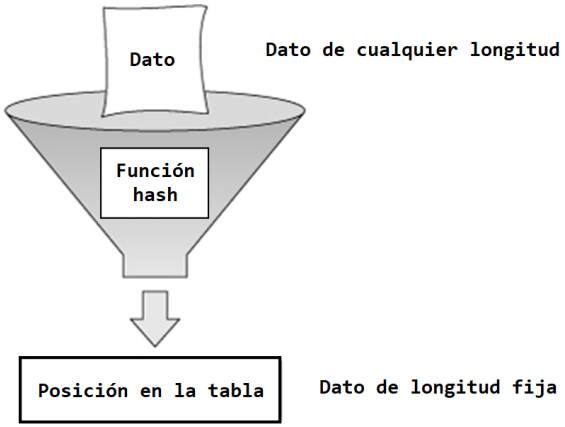
Implementación mediante listas

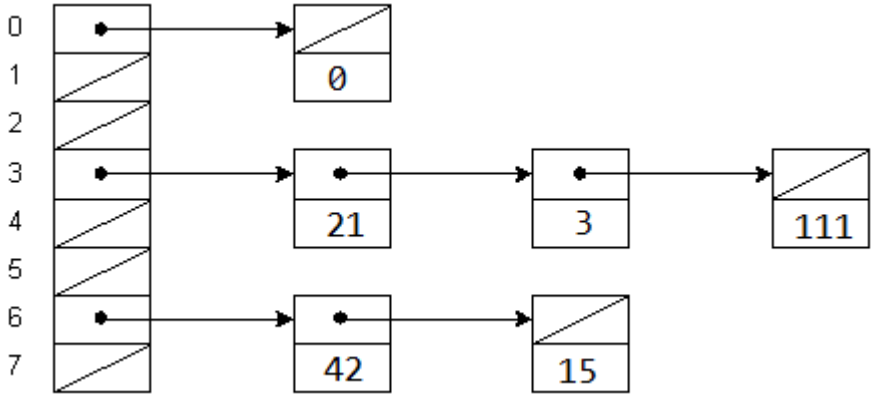
- Se utiliza una lista ordenada (o no) para almacenar los valores del conjunto.
  - Todo elemento en la lista, está en el conjunto
  - En bolsas, hay un campo adicional para la multiplicidad



Implementación mediante tablas hash

- Una función hash facilita un acceso eficiente a la lista de claves
- Como puede haber colisiones (elementos con la misma clave hash) en cada valor de la clave hay una lista enlazada con todos los valores cuyo hash es el mismo.





Conjuntos en Pascal

- En Pascal el tipo **`Set`** permite conjuntos homogéneos cuyo tipo base es cualquier ordinal (incluidos enumerados)
- Todo conjunto en Pascal está limitado a 256 elementos

Operación	Operador
Unión	`+`
Intersección	`*`
Diferencia	`-`
Diferencia simétrica	`><`
Igualdad	`=`
Desigualdad	`<>`
Inclusión	`<= , >=`
Pertenencia	`in`
Añadir/eliminar elementos	`include / exclude`

Conjuntos en Pascal

- Operaciones básicas:

```
type
    tipo_base = (value1, value2, ... value_n);
    tSet = set of tipo_base;

var
    my_set: tSet;

begin
    my_set := [value_i, value_j...];
    if some_value in my_set then do_something
end.
```

(Como tipo base sirve cualquier ordinal cuyo rango de valores sea máximo 256)

Conjuntos en Pascal

- Ejemplo:

```
type
    language = (solidity, java, python, C++, ruby, go);
    languages = set of language;

var
    programmer_skills: languages;

begin
    programmer_skills := [java, python];
    if solidity in programmer_skills then
      writeLn(‘The guy is a blockchain developer!');
end.
```

Conjuntos en Pascal

- Ejemplo:

```
type
  set_caracteres = set of char;

var
  s1, s2: set_caracteres;
begin
  s1 := ['1', '2','3','4'];
  s2 := ['3','4','5','6'];
  include(s1,'X');
  exclude(s1,'2');
  writeln('X' in s1);
  readln;
end.
```

Ejercicios propuestos

Ejercicio propuesto 1

Utilizando un conjunto de Pascal, añade una operación al TAD ListaEnteros que permita eliminar todos los valores duplicados de una lista.

No es necesario leer la entrada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(n)

Ejercicio propuesto 1bis

Repite el ejercicio anterior aplicando esta vez la implementación al TAD ListaGenérica y comenta qué modificaciones serían necesarias.

No es necesario leer la entrada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(n)

Ejercicio propuesto 2

Implementa un TAD `ConjuntoStrings` donde el tipo base serán cadenas de texto de cualquier longitud.

Programa las operaciones básicas esenciales del TAD en una Unit de Pascal utilizando una lista ordenada como representación interna.

Ejercicio propuesto 3

Implementa las operaciones usuales entre conjuntos (Unión, Intersección, Diferencia e Inclusión) sobre el TAD `ConjuntoStrings` programado en el ejercicio 2

Ejercicio propuesto 4 (reto)

Implementa el TAD `BolsaStrings` similar a los TADs programados en los ejercicios anteriores, pero utilizando como representación interna una tabla hash. Deberás programar también la función hash, para lo cual deberás investigar un poco al respecto.